

Dokumentacja Projektu: scierzko-znajdywacz

1. Architektura Systemu i Wzorce Projektowe

Projekt został zrealizowany w oparciu o architekturę **MVC (Model-Widok-Kontroler)**, co zapewnia ścisłą separację logiki biznesowej od technologii renderowania interfejsu użytkownika (Qt6). Dodatkowo w kodzie zastosowano fundamentalne paradygmaty programowania obiektowego (OOP):

- **Abstrakcja i Polimorfizm:** Klasa `AlgorytmSciezki` definiuje abstrakcyjny interfejs dla wszystkich mechanizmów wyszukiwania. Kontroler (`OknoGlowne`) zarządza algorytmami polimorficznie za pomocą wskaźnika klasy bazowej `AlgorytmSciezki*`, co umożliwia dynamiczną wymianę strategii wyszukiwania w czasie wykonywania programu.
- **Hermetyzacja (Kapsułkowanie):** Klasa `Plansza` w pełni ukrywa swoje wewnętrzne struktury danych (takie jak dwuwymiarowy wektor reprezentujący siatkę). Modyfikacja i odczyt stanu komórek odbywają się wyłącznie poprzez publiczne metody dostępowe (getter/setter), które walidują poprawność współrzędnych.
- **Separacja Logiki:** Komponenty odpowiedzialne za przechowywanie danych (`Plansza`, `Punkt`) oraz ich przetwarzanie (`AlgorytmSciezki` i klasy pochodne) są całkowicie pozbawione zależności od biblioteki Qt, dzięki czemu logikę biznesową można testować w czystym środowisku konsolowym.

2. Szczegółowy Opis Struktury i Modelu Danych (Model)

2.1. Struktura Punkt (include/punkt.h)

Lekka struktura reprezentująca współrzędne dwuwymiarowe komórki na planszy. Zastępuje standardowy szablon `std::pair<int, int>`, zwiększając czytelność kodu źródłowego.

- **Pola składowe:**
 - `int wiersz` – indeks osi pionowej (indeksowany od 0 od góry do dołu).
 - `int kolumna` – indeks osi poziomej (indeksowany od 0 od lewej do prawej).
- **Konstruktor:** Domyślny (zerujący współrzędne do wartości 0, 0) oraz parametryczny `Punkt(int w, int k)`.
- **Przeładowane operatory:** operator `==` oraz operator `!=` umożliwiające bezpośrednie porównywanie pozycji dwóch punktów w przestrzeni dyskretnej.

2.2. Typ Wyliczeniowy TypKomorki (include/plansza.h)

Silnie typowane wyliczenie (enum class) określające stan fizyczny oraz logiczny pojedynczego pola na siatce:

- Pusta – pole wolne, po którym algorytmy mogą się poruszać (domyślny stan tła planszy).
- Sciana – nieprzejezdna przeszkoda; pole blokujące ruch algorytmów.
- Odwiedzona – pole zbadane (przetworzone) przez dany algorytm w fazie przeszukiwania przestrzeni.
- Ścieżka – pole wchodzące w skład wyznaczonej, optymalnej trasy końcowej od punktu startowego do celu.

2.3. Klasa Plansza (include/plansza.h, src/plansza.cpp)

Klasa zarządzająca stanem siatki grafu dyskretnego.

- **Wewnętrzna struktura:** Stan planszy przechowywany jest w prywatnym polu siatka, będącym dwuwymiarowym wektorem dynamicznym: `std::vector<std::vector<TypKomorki>>`.
- **Kluczowe metody:**
 - `Plansza(int w, int k)` – konstruktor alokujący pamięć dla siatki o wymiarach $w \times k$ i wypełniający ją wartościami `TypKomorki::Pusta`. Inicjalizuje punkt startowy w lewym górnym rogu $(0, 0)$, a cel w prawym dolnym rogu $(w-1, k-1)$.
 - `wyczyszcWszystko()` – resetuje całą matrycę do stanu `Pusta` oraz przywraca domyślne, startowe pozycje punktów węzłowych.
 - `wyczyszcSciezkeIOdwiedzone()` – iteruje po strukturze danych i przywraca stan `Pusta` wszystkim komórkom oznaczonym jako `Odwiedzona` lub `Ścieżka`. Metoda ta zachowuje nienaruszoną strukturę ścian (barier).
 - `wyczyszcSciany()` – usuwa z planszy wyłącznie przeszkody (`Sciana`), pozostawiając ślady algorytmów bez zmian.
 - `czyWGranicach(int w, int k)` – weryfikuje warunki brzegowe: $0 \leq w < \text{wiersze}$ oraz $0 \leq k < \text{kolumny}$. Zapobiega błędowi naruszenia pamięci (out-of-bounds).
 - `czyMoznaWejsc(Punkt p)` – sprawdza, czy dany punkt leży w granicach planszy oraz czy jego typ jest różny od `TypKomorki::Sciana`.

3. Dogłębna Analiza Algorytmów Wyszukiwania Ścieżki

Wszystkie algorytmy operują na grafie nieskierowanym o strukturze siatki, gdzie krawędzie łączą sąsiadujące komórki w czterech podstawowych kierunkach (góra, dół, lewo, prawo). Przejście między sąsiadami ma jednolity koszt równy 1.

3.1. Struktura WynikWyszukiwania (include/algorytm.h)

Kontener przesyłowy zwracany przez silnik obliczeniowy:

- `std::vector<Punkt> kolejnoscOdwiedzania` – chronologiczna lista współrzędnych przetwarzanych przez algorytm (niezbędna do odtworzenia animacji badania grafu krok po kroku).
- `std::vector<Punkt> najkrotszaSciezka` – sekwencja punktów stanowiąca rozwiązanie problemu poszukiwania drogi.
- `bool znaleziono` – flaga informująca o istnieniu ścieżki łączącej węzły.

3.2. Przeszukiwanie Wszerz (BFS - AlgorytmBFS)

Algorytm eksploracji grafu wykorzystujący strukturę kolejki FIFO (`std::queue`).

Gwarantuje znalezienie ścieżki o minimalnej liczbie krawędzi w grafach nieważonych.

- **Zasada działania:** Dodaje punkt startowy do kolejki i oznacza jako odwiedzony. W pętli pobiera element z czoła kolejki, po czym analizuje jego 4 sąsiadów. Każdy nieodwiedzony i przejezdny sąsiad jest dodawany do kolejki, a jego relacja z rodzicem jest zapisywana w macierzy `rodzic`.
- **Odtwarzanie ścieżki:** Po natrafieniu na węzeł docelowy, algorytm wykonuje pętlę wsteczną po macierzy `rodzic` aż do osiągnięcia punktu startowego, po czym odwraca wektor wynikowy za pomocą `std::reverse`.

3.3. Przeszukiwanie w Głęb (DFS - AlgorytmDFS)

Algorytm oparty na strategii LIFO (Last In First Out) zaimplementowany przy użyciu `std::vector` działającego jako stos.

- **Charakterystyka obliczeniowa:** DFS nie gwarantuje wyznaczenia najkrótszej drogi. Przeszukuje on wybraną ścieżkę do momentu napotkania ślepego zaułka, zanim zacznie się cofać (backtracking). Powoduje to, że generowana trasa często posiada nienaturalne pętle i załamania.

3.4. Algorytm A* (AlgorytmAStar)

Heurystyczny algorytm wyszukiwania najkrótszej ścieżki optymalizujący całkowity koszt drogi.

- **Funkcja oceny:** Definiowana wzorem $F(n) = G(n) + H(n)$, gdzie:
 - $G(n)$ – rzeczywisty koszt dotarcia ze startu do węzła n .
 - $H(n)$ – szacowany koszt (heurystyka) z węzła n do celu.
- **Heurystyka:** Wykorzystano odległość Manhattan, obliczaną jako:
 $Manhattan(a, b) = |a.wiersz - b.wiersz| + |a.kolumna - b.kolumna|$

Jest to heurystyka dopuszczalna (nie przeszacowuje rzeczywistego kosztu) i spójna dla siatek o ruchu czterokierunkowym.

- **Struktura WezelAStar:** Przechowuje pozycję, G oraz H. Posiada przeciążony operator > służący do sortowania wewnątrz `std::priority_queue`. Dzięki przekazaniu szablonu `std::greater`, kolejka priorytetowa zachowuje się jak kopiec typu min (węzeł o najniższej wartości $F(n)$ znajduje się zawsze na szczycie).

3.5. Algorytm Dijkstry (AlgorytmDijkstry)

Klasyczny algorytm wyznaczania najkrótszych ścieżek z jednego źródła. W tej konkretnej implementacji (siatka o jednolitych wagach krawędzi równych 1) działa on analogicznie do algorytmu BFS, lecz zarządza węzłami przy użyciu kolejki priorytetowej dystansów (`WezelDijkstry`). Reprezentuje gotową bazę pod rozbudowę systemu o siatki o zróżnicowanym koszcie przejścia (np. teren trudny, błoto, drogi).

4. Warstwa Graficzna i Mechanizm Animacji (Widok)

4.1. Klasa WidokPlanszy (include/widok_planszy.h, src/widok_planszy.cpp)

Niestandardowy komponent graficzny Qt dziedziczący po `QWidget`, odpowiedzialny za rysowanie stanu modelu oraz przechwytywanie interakcji myszy.

A. Cykl Życia Animacji

1. Kontroler wywołuje metodę `uruchomWizualizacje(const WynikWyszukiwania& wynik, int predkoscMs)`.
2. Struktury wynikowe są kopiowane do buforów animacji: `animacjaOdwiedzone` oraz `animacjaSciezka`, a indeksy klatek są zerowane.
3. Uruchamiany jest stoper `QTimer` z interwałem `predkoscMs`.
4. W każdym cyklu stopera wywoływany jest slot `krokAnimacji()`:
 - a. **Faza Badania:** Dopóki `indeksOdwiedzonych < animacjaOdwiedzone.size()`, system pobiera punkt z bufora, zmienia jego stan w modelu na `TypKomorki::Odwiedzona` i wymusza odświeżenie ekranu poprzez `update()`.
 - b. **Faza Rysowania Ścieżki:** Po zakończeniu pierwszej fazy, system sekwencyjnie pobiera punkty z `animacjaSciezka`, ustawia ich typ na `TypKomorki::Sciezka` i wywołuje `update()`.
 - c. **Faza Końcowa:** Po wyczerpaniu obu buforów stoper zostaje zatrzymany, flaga `wTrakcieAnimacji` przyjmuje wartość `false`, a komponent emituje sygnał `koniecWizualizacji(dlugoscSciezki, liczbaOdwiedzonych)` do kontrolera.

B. Mechanizm Renderowania (paintEvent)

Rysowanie odbywa się z wykorzystaniem silnika QPainter. Włączany jest antyaliasing (`QPainter::Antialiasing`) w celu wygładzenia krawędzi ikon startu i celu.

- **Dynamiczne Skalowanie:** Szerokość (`szerokoscKomorki`) i wysokość (`wysokoscKomorki`) pojedynczego pola obliczane są na bieżąco na podstawie aktualnych wymiarów widżetu (`width()` i `height()`) oraz wymiarów planszy. Zapewnia to pełną responsywność interfejsu przy zmianie rozmiaru okna programu.
- **Geometria Węzłów Specjalnych:**
 - **Start:** Rysowany jako jasnozielona strzałka skierowana w prawo przy użyciu struktury wielokąta `QPolygon` o sztywno zdefiniowanych proporcjach wektora przesunięcia względem prostokąta komórki.
 - **Cel:** Rysowany jako czerwona tarcza strzelecka. Składa się z zewnętrznego okręgu bez wypełnienia (grubość linii: 2 piksele) oraz wewnętrznego, pełnego koła, uzyskanego dzięki metodzie `QRect::adjusted()` zmniejszającej promień rysowania.

C. Maszyna Stanów Interakcji Myszy

Widżet implementuje interaktywne edytowanie planszy w trzech przeciążonych metodach Qt:

- `mousePressEvent` – Wyznacza współrzędne pola na podstawie pozycji kursora ($\$x / \text{\texttt{\texttt{szerokoscKomorki}}}$, $\$y / \text{\texttt{\texttt{wysokoscKomorki}}}$). Jeżeli kliknięto na węzeł startowy lub docelowy, aplikacja przechodzi odpowiednio w stan `przeciąganieStartu` lub `przeciąganieCelu`. W przeciwnym razie sprawdzany jest stan pola – jeśli nie jest to ściana, ustawiany jest tryb rysowania ścian, w przeciwnym razie tryb ich usuwania.
- `mouseMoveEvent` – Działa dwutorowo. Jeśli aktywna jest flaga przeciągania węzłów, aktualizuje pozycję startu/celu w modelu (uniemożliwiając nałożenie startu na cel i odwrotnie). Jeśli użytkownik przesuwaa mysz z wciśniętym lewym przyciskiem poza węzłami, funkcja masowo rysuje bądź usuwa ściany wzdłuż ścieżki ruchu kursora.
- `mouseReleaseEvent` – Resetuje stany przeciągania elementów po puszczeniu przycisku myszy.

5. Kontroler i Integracja Interfejsu (Kontroler)

5.1. Klasa OknoGlowne (`include/okno_glowne.h`, `src/okno_glowne.cpp`)

Klasa zarządzająca oknem głównym aplikacji (`QMainWindow`), odpowiedzialna za kompozycję interfejsu oraz dystrybucję zdarzeń.

A. Układ Przestrzenny (Layout Management)

Interfejs podzielony jest w pionie za pomocą kontenera `QVBoxLayout` na cztery główne warstwy funkcjonalne:

1. **Górny Pasek (Navbar):** Poziomy układ `QHBoxLayout` grupujący nagłówek tekstowy, kontrolki wyboru (`QComboBox`) dla algorytmów i prędkości animacji oraz zestaw przycisków akcji (`QPushButton`). Element `addStretch()` spycha komponenty do lewej krawędzi.
2. **Pasek Legendy (LegendContainer):** Wyśrodkowany horyzontalnie panel, który dynamicznie alokuje obiekty pomocnicze `LegendItem` objaśniające znaczenie kolorów siatki.
3. **Centralny Widżet Planszy (widokPlanszy):** Dodany do layoutu głównego z wagą rozciągania 1, co sprawia, że konsumuje on całą pozostałą przestrzeń okna.
4. **Dolny Pasek Stanu (Footer):** Etykieta `QLabel` wyświetlająca bieżące instrukcje lub statystyki końcowe pracy algorytmów.

B. Polimorficzna Logika Uruchamiania Algorytmów

Wewnątrz metody `kliknietoWizualizuj()` realizowane jest kluczowe powiązanie warstw:

C++

```
AlgorytmSciezki* algorytm = nullptr;

if (nazwaAlgorytmu == "Przeszukiwanie Wszerz (BFS)") {
    algorytm = new AlgorytmBFS();
} else if (nazwaAlgorytmu == "Algorytm A*") {
    algorytm = new AlgorytmAStar();
} else if (nazwaAlgorytmu == "Przeszukiwanie w Głębokość (DFS)") {
    algorytm = new AlgorytmDFS();
} else if (nazwaAlgorytmu == "Algorytm Dijkstry") {
    algorytm = new AlgorytmDijkstry();
}
```

Wskaźnik `algorytm` typu bazowego dynamicznie przyjmuje adres obiektu instancji klasy pochodnej. Wywołanie `algorytm->szukaj(plansza, ...)` realizuje dynamiczne wiązanie metod (polimorfizm). Po pobraniu wyników i przekazaniu ich do widoku planszy, pamięć przydzielona na `algorytm` jest bezzwłocznie zwalniana za pomocą operatora `delete`, eliminując wycieki pamięci.

C. Warstwa Stylów CSS (Qt Stylesheet)

Wygląd aplikacji zdefiniowany jest wewnątrz funkcji `pobierzStyleAplikacji()` przy użyciu składni wzorowanej na CSS. Zaimplementowano nowoczesny, ciemny motyw dla paska nawigacji (`#Navbar` z tłem `#1a202c`), zaokrąglone krawędzie dla kontrolek (`border-radius: 5px`) oraz dedykowane pseudostany `:hover` i `:pressed` podnoszące responsywność wizualną przycisków. Akcje destrukcyjne, jak pełny reset planszy, wyróżnione są kolorem czerwonym (`#e53e3e`).

6. Tryb Konsolowy i Moduł Diagnostyczny

Aplikacja posiada wbudowany, niezależny moduł diagnostyczny `TesterKonsolowy`, sprawdzający poprawność działania silnika matematycznego bez ładowania podsystemu graficznego okien.

6.1. Klasa `TesterKonsolowy` (`src/tester_konsolowy.cpp`)

- Metoda `uruchomTest()` buduje izolowany obiekt planszy o wymiarach 12×12 komórek.
- Ręcznie programuje pozycje startu $(1, 1)$, celu $(10, 10)$ oraz buduje ciągłą ścianę blokującą bezpośrednią trasę po przekątnej.
- Za pomocą strumienia `std::cout` renderuje do terminala mapę ASCII, gdzie poszczególne symbole oznaczają odpowiednio: S – start, C – cel, # – ściana, . – wolne pole.
- Uruchamia Algorytm Dijkstry, pobiera strukturę wyników, oznacza wyznaczoną trasę symbolem x i ponownie drukuje stan planszy wraz z parametrami liczby kroków oraz długości ścieżki.

6.2. Funkcja Główna `main` (`src/main.cpp`)

Plik `main.cpp` steruje trybem uruchomienia programu. Analizuje tablicę argumentów wejściowych linii komend (`argv`). Jeżeli wykryty zostanie którykolwiek z argumentów: `--konsola`, `--test` lub `--console`, aplikacja uruchamia procedurę `TesterKonsolowy::uruchomTest()`, zwraca kod 0 i kończy działanie przed utworzeniem obiektu `QApplication`. Zapewnia to pełną wielozadaniowość binariów na serwerach pozbawionych serwera środowiska graficznego (np. systemy CI/CD).

7. Analiza Konfiguracji Systemu Budowania (CMake)

Plik `CMakeLists.txt` zarządza procesem kompilacji i linkowania bibliotek zewnętrznych.

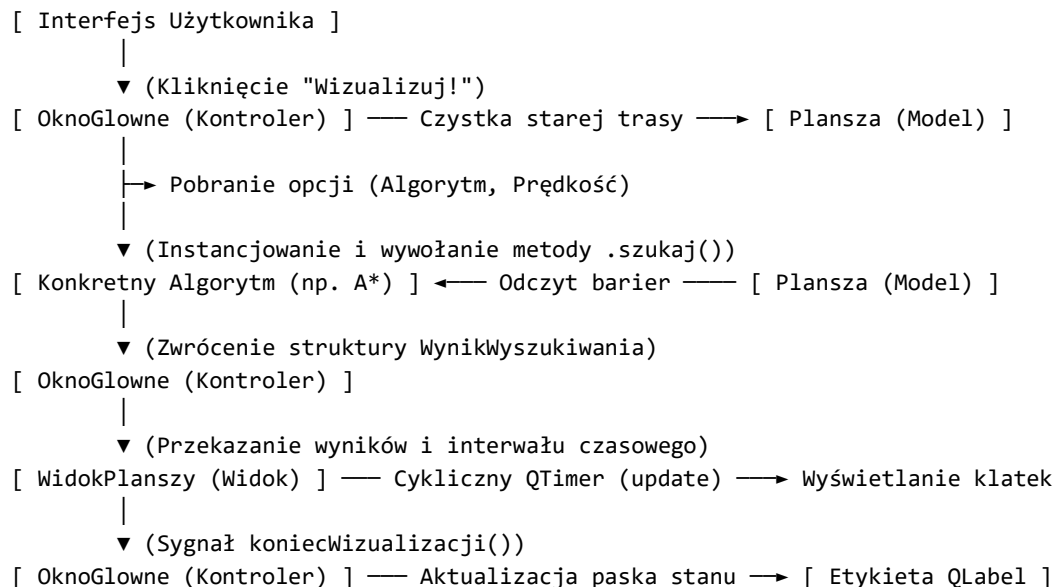
- **Wymagania i Standardy:** Minimalna wersja narzędzia CMake ustawiona jest na 3.30, a standard języka kompilatora na C++20. Dodatkowo włączono optymalizację międzyproceduralną (LTO/IPO) poprzez ustawienie flagi

CMAKE_INTERPROCEDURAL_OPTIMIZATION na TRUE, co pozwala na optymalizację kodu w fazie linkowania.

- **Automatyzacja Qt:** Flagi CMAKE_AUTOMOC, CMAKE_AUTOUIC oraz CMAKE_AUTORCC są włączone (ON). Informują one CMake o konieczności automatycznego skanowania plików źródłowych i uruchamiania generatorów meta-obiektów Qt w tle.
- **Zarządzanie plikami:** Funkcja `file(GLOB_RECURSE MY_SOURCES ...)` przeszukuje rekurencyjnie podkatalog `src/` w poszukiwaniu plików `.cpp`. Dyrektywa `CONFIGURE_DEPENDS` instruuje system budowania, by automatycznie ponownie uruchomił konfigurację CMake, jeśli w katalogu pojawią się nowe pliki źródłowe, eliminując potrzebę ręcznego resetowania projektu przez dewelopera.
- **Weryfikacja Flag:** Zaimplementowano niestandardową funkcję makra `enable_cxx_compiler_flag_if_supported`, która bezpiecznie weryfikuje, czy dany kompilator (np. GCC, Clang, MSVC) wspiera flagę restrykcyjnych ostrzeżeń - `Wall`, zapobiegając przerwaniu procesu generowania skryptów na niewspieranych platformach.

8. Przepływ Danych w Aplikacji (Data Flow Diagram)

Poniższa sekwencja obrazuje kompletną drogę danych od kliknięcia użytkownika do wyświetlenia wyników:



Ta architektura gwarantuje stabilność, łatwość wprowadzania modyfikacji deweloperskich oraz brak bezpośrednich sprzężeń zwrotnych pomiędzy odległymi warstwami kodu źródłowego.

